

CMake Presets

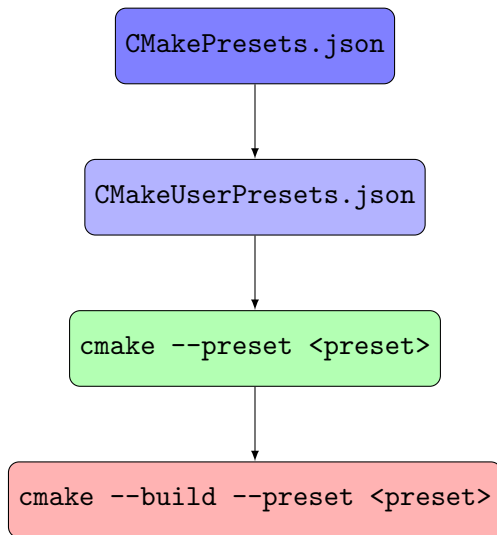
Simplification et Optimisation des Configurations de Build

- CMake Presets : une solution pour gérer efficacement les configurations multiples (CUDA, HIP, Kokkos, Raja, Debug, ...)
- Centralisation des configurations dans des fichiers JSON : `CMakePresets.json` et `CMakeUserPresets.json`.

Pourquoi Utiliser les CMake Presets ?

- Les CMake Presets offrent une gestion plus "user-friendly" des configurations et du développement de quelconques projets.
- Centralisation des configurations pour une meilleure reproductibilité.
- Les presets permettent une meilleure flexibilité pour basculer d'une configuration à l'autre.
- Facilité d'utilisation des presets avec un IDE.

Comment Utiliser les CMake Presets ?



Introduction à CMakePresets.json

- **Objectif** : Centraliser les configurations de build avec CMake.
- **Structure** : Deux sections principales pour gérer les configurations et les builds.
- **Spécificité de la machine** : L'utilisateur crée le preset adapté à sa machine $\implies$ chemin vers les compilateurs, architecture cuda, ...
Cette partie peut être aussi bien indiquée dans la partie

CMakeUserPresets.json

- **Objectif** : Définir les configurations initiales du projet.
- **Champs clés** :
 - **name** : Nom du preset de configuration.
 - **generator** : Générateur de build (ex. : Ninja, Makefiles).
 - **binaryDir** : Répertoire de build.
 - **cacheVariables** : Variables de cache(ex. :CMAKE_C_COMPILER, Flags de compilation)

Visualisation d'un configurePreset

```
1  {
2    "version": 3,
3    "cmakeMinimumRequired": {
4      "major": 3,
5      "minor": 21,
6      "patch": 0
7    },
8    "configurePresets": [
9      {
10       "name": "default",
11       "displayName": "Default Config",
12       "description": "Default config",
13       "generator": "Unix Makefiles",
14       "binaryDir": "${sourceDir}/build/default",
15       "cacheVariables": {
16         "CMAKE_CXX_COMPILER": "g++",
17         "CMAKE_C_COMPILER": "gcc",
18         "CMAKE_INSTALL_PREFIX": "${sourceDir}/build/default/install/",
19         "CMAKE_BUILD_TYPE": "Release",
20         "USE_OMP": "ON",
21         "USE_RAJA": "OFF",
22         "USE_KOKKOS": "OFF",
23         "ENABLE_TESTS": "ON"
24       },
25       "vendor": {
26         "example.com/ExampleIDE/1.0": {
27           "autoFormat": true
28         }
29       },
30     },
31     {
32       "name": "kokkos",
33       "displayName": "kokkos Config",
34       "description": "kokkos config",
35       "generator": "Unix Makefiles",
36       "binaryDir": "${sourceDir}/build/kokkos",
37       "cacheVariables": {
38         "CMAKE_CXX_COMPILER": "g++",
39         "CMAKE_C_COMPILER": "gcc",
40         "CMAKE_INSTALL_PREFIX": "${sourceDir}/build/kokkos/install/",
41         "CMAKE_BUILD_TYPE": "Release",
42         "USE_OMP": "OFF",
43         "USE_RAJA": "OFF",
44         "USE_KOKKOS": "ON",
45         "ENABLE_TESTS": "ON"
46       },
47       "vendor": {
48         "example.com/ExampleIDE/1.0": {
49           "autoFormat": true
50         }
51       },
52     }
53   ],
54 }
```

- **Objectif** : Définir les configurations spécifiques à la construction.
- **Champs clés** :
 - **name** : Nom du preset de build.
 - **configurePreset** : Référence au preset de configuration.
 - **configuration** : Type de build (ex. : Release, Debug).
 - **jobs** : Nombre de jobs parallèles.
 - **targets** : Cibles à construire (ex. : all).

Visualisation d'un buildPreset

```
54     "buildPresets": [  
55         {  
56             "name": "default",  
57             "configurePreset": "default",  
58             "jobs": 10  
59         },  
60         {  
61             "name": "kokkos",  
62             "configurePreset": "kokkos",  
63             "jobs": 10  
64         }  
65     ],
```

- **Définir les Tests :**

- `enable_testing()` dans le `CMakeLists.txt` pour réaliser des tests.
- Ajout de tests avec `add_test()`.

- **Configurer les Presets :**

- **Output** : Options de sortie pour contrôler l'affichage des résultats des tests.
- **Execution** : Action à faire ou non en cas de réussite ou d'échec à l'exécution
- **Environment** : Spécifie les variables d'environnement nécessaires pour l'exécution des tests.

Visualisation d'un testPreset

```
66     "testPresets": [  
67       {  
68         "name": "default",  
69         "configurePreset": "default",  
70  
71         "output": {  
72           "outputOnFailure": true  
73         },  
74         "execution": {  
75           "noTestsAction": "ignore",  
76           "stopOnFailure": true  
77         },  
78         "environment": {  
79           "OMP_PROC_BIND": "true"  
80         }  
81       },  
82       {  
83         "name": "kokkos",  
84         "inherits": "default",  
85         "configurePreset": "kokkos"  
86       }  
87     ],  
88     "vendor": {  
89       "example.com/ExampleIDE/1.0": {  
90         "autoFormat": false  
91       }  
92     }  
93   }
```

Petit exemple

Exemple de cmake preset

Presets pour la Proxyapp

Objectif :

- Simplifier la construction de la Proxyapp :

```
cmake -DCMAKE_BUILD_TYPE=RELEASE  
-DUSE_RAJA=ON  
-DBUILD_FROM_TPLMIRROR=OFF  
-C configs/config_proxy-app.cmake  
-B ./build -DCMAKE_INSTALL_PREFIX=./install -S .
```

⇓ **Simplification avec un preset**

```
cmake --preset raja_no_tpl_mirror
```

- L'utilisateur n'a plus à modifier les fichiers `config_<architecture>.cmake`
- Modification directement dans `CMakeUserPresets.json`

- Fichier CMakePreset.json $\implies$ toutes les presets possibles pour toutes les options (Kokkos, Kokkos+GPU, Kokkos+GPU+openMP, Raja,TPLMirror, etc) avec aucune spécificité liée à l'environnement
- CMakeUserPreset.json $\implies$ 3 presets à modifier par l'utilisateur selon besoin :
 - Preset correspondant au début de la partie 2 du fichier `config_<architecture>.cmake` : CXX_ROOT, CPU_TUNE_FLAG, etc
 - Preset spécifique à CUDA ou HIP : PATH, Architecture,etc
- **⚠ ⚠** accès à l'utilisateur de preset non valide sur son architecture : Cuda sur architecture AMD et HIP sur architecture Nvidia

- Travail identique pour les TPLs $\implies$ l'utilisateur modifie seulement les 2 presets spécifiques à sa machine $\implies$ `cmake --preset kokkos-cuda`
- Permet de tester l'utilisation de flags (réflexion de Félix) : celui-ci est inclus dans un preset qu'il est possible de cacher (n'apparaît pas lors de `cmake --list-presets` pour l'utilisateur, pour la phase de développement le remettre visible via `"hidden": true`)
- Génération des `CMakeUserPreset.json` via script $\implies$ permet d'éliminer les presets non adaptés à la machine